



**Department of Electrical,
Computer, & Biomedical Engineering**
Faculty of Engineering & Architectural Science

Course Title:	COE891
Course Number:	Software Testing and QA
Semester/Year (e.g.F2016)	W2024

Instructor:	Samavi
-------------	--------

Assignment/Lab Number:	1
Assignment/Lab Title:	JUnit: Automated Testing

Submission Date:	01/29/2024
Due Date:	01/29/2024

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Nagra	Navjeev	500969113	3	N.N.

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/current/pol60.pdf>

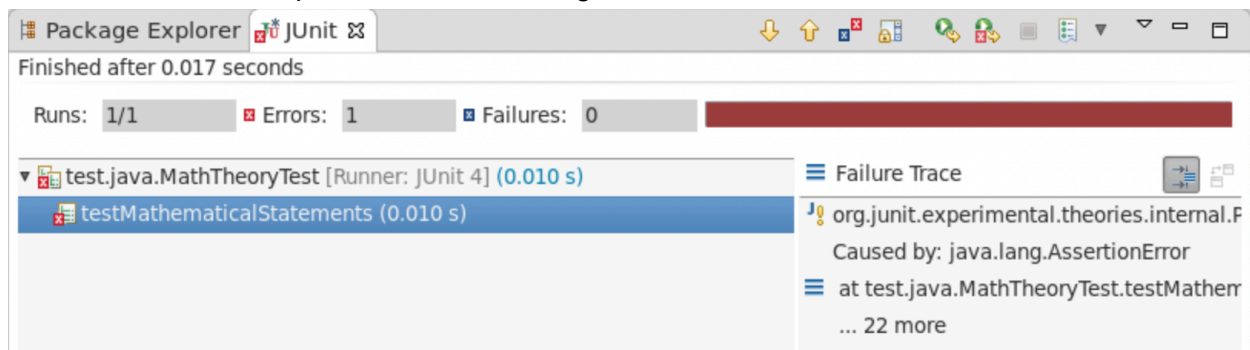
Lab Questions

Q2. If a user creates a triangle with dimensions 3, 4 and 100, it is apparent this triangle is an invalid triangle. This is testable in JUnit, as the program should throw an `IllegalArgumentException` in this case, however `Triangle.java` does not have any logic to detect an invalid triangle. The JUnit test case for the invalid triangle case correctly expects an `IllegalArgumentException`, however it is not thrown. So, the test fails. If some code were added to the constructor to detect an invalid triangle, the test would pass.

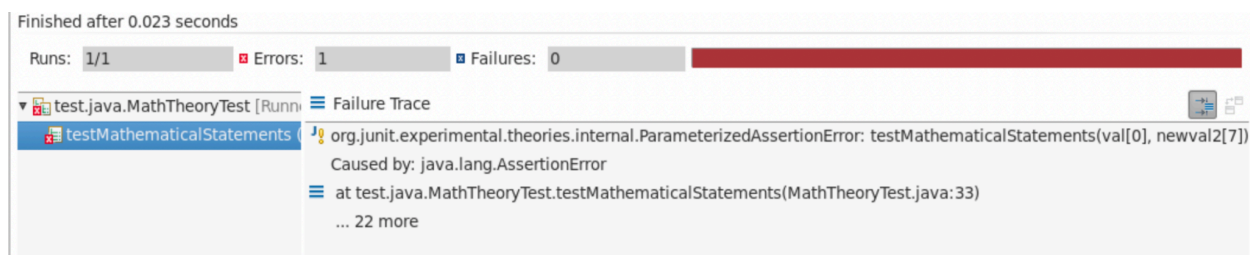
Q3. When creating a string, “\d” is an expression character class that matches digits from 0-9. It represents a range of digits. For example, `d{3}` in the `RE.java` class needs 3 digits, which are the first 3 digits of a phone number.

When testing the expression with `(123)123 - 1234`, `(123) 456 - 7890` and `123 123 - 1234`, their respective tests were failing. This is because the original regex did not escape the parentheses within the expression, so the tests failed.

Q7. When using the `newVal` set, there are negative values present (`-1`, `-10`, `-1234`). The statements `a+b > a` and `a+b > b` only hold true for all `a, b > 0`. Therefore, the test case will fail, as there are no assumptions made in the original test case. The failure can be seen below:



When considering the new value set with `Integer.MAX_VALUE` and `Integer.MIN_VALUE`, the test case should fail. This is because when a value is added to `MAX_VALUE`, it will cause an integer overflow, and will fail the test. The results are consistent with my analysis:



The issue is with `newVal2[7]`, which is the element containing `INTEGER.MAX_VALUE`. If the data type were set to `long`, or some additional assumptions were made, the test would pass.